예측시장 데이터 파이프라인 학습서

Contents

1 예측시장 데이터 파이프라인이 푸는 문제 **1**
- 1.1 예측시장이란 무엇인가: Kalshi 와 Polymarket 은 같은 제품군이다 1
- 1.2 두 거래소, 다른 배선 2
- 1.3 왜 수집이 어려운가: 델타, 재생 불가, 시계 불일치 3
- 1.4 BTC 한 종목으로 시작하는 이유 5
- 1.5 시스템의 목적: " 적어도 한 번" 에서" 정확히 한 번" 으로 5

2 왜 Kafka 인가, 왜 Redis 와 ClickHouse 둘 다인가 **7**
- 2.1 먼저 실패하는 설계: 전부 Redis 에 담으면 어떻게 되는가 7
- 2.2 Redis 는 현재 상태, ClickHouse 는 진실 8
- 2.3 Kafka 가 그 사이에 있는 이유 9
- 2.4 왜 흐름을 먼저 mock 으로 증명하는가 11
- 2.5 정리: 이 장이 다음 장으로 넘기는 것 12

3 스키마를 계약으로 두는 설계 **13**
- 3.1 계약이 세 층인 이유 13
- 3.2 와이어 포맷이 JSON 인 이유, 그리고 이것이 잠정 결정인 이유 . . . 14
- 3.3 규칙이 조용히 깨지는 세 지점 15
- 3.4 한 단어가 세 가지 뜻을 지고 있었다: `venue`, `ingest_path`, `feed` 16
- 3.5 `venue` 가 정렬 키 선두에 서는 이유 17
- 3.6 와이어 포맷이 바뀌어도 형태의 원본은 proto 다 18
- 3.7 파생 타입은 파생 원본 옆에 산다 19
- 3.8 정리: 계약 하나가 지나가는 세 층 20

4 델타에서 스냅샷을 재구성한다는 것 **21**

5 at-least-once 에서 exactly-once 로 **23**
- 5.1 배경: " 적어도 한 번" 과" 정확히 한 번" 이라는 말이 왜 미끄러운가 23
- 5.2 앱 내부 큐: at-least-once 조차 아닌 구간 24
- 5.3 브로커부터 ClickHouse 까지: at-least-once 가 성립하는 구간 . . . 25

5.4 파이프라인 전체 지도 26
5.5 DB 단에서 exactly-once 로 접는 방법 26
5.6 왜 Kafka 트랜잭션 (EOS) 이 아니라 DB 중복 제거인가 28
5.7 정직하게 짚는 잔존 위험 28
5.8 한 줄로 보는 경로별 요약 29

6 예측과 트레이딩 **31**

Chapter 1

예측시장 데이터 파이프라인이 푸는 문제

In brief.

Why Kalshi, Polymarket 같은 예측시장 (prediction market) 과 Binance, Coinbase 같은 현물 거래소 (spot exchange) 에서 BTC 관련 데이터를 실시간으로 모으는 일은 API 를 반복 호출하는 문제가 아니다. 피드가 끊기고, 메시지가 중복되고, 순서가 뒤집히는 조건에서도 하나의 오더북 (orderbook, 호가창) 상태를 일관되게 유지해야 한다는 요구가 있다.

When 이 장의 논의는 producer(거래소 WS 를 구독해 데이터를 내보내는 프로세스) 가 켜지는 순간부터 대시보드에 값이 표시되는 순간까지, 파이프라인 전 구간에 적용된다.

Where Kalshi·Polymarket(예측시장) 및 Binance·Coinbase(현물) 의 BTC 관련 데이터에 적용된다. 이 문서가 다루는 파이프라인은 BTC 한 종목으로 범위를 좁혀 시작한다 (절 1.4).

How " 왜 어려운가" 를 먼저 확인하고, 그 답이 장 2(토폴로지), 장 3(스키마), 장 5(전달 보장) 의 설계를 낳는 순서로 이 책 전체를 읽는다.

1.1 예측시장이란 무엇인가: Kalshi 와 Polymarket 은 같은 제품군이다

무슨 문제가 있었나. Kalshi 와 Polymarket 은 겉으로 보면 전혀 다른 물건처럼 보인다. Kalshi 는 미국 상품선물거래위원회 관할 아래 인증을 요구하는 거래소이고, Polymarket 은 누구나 인증 없이 접속할 수 있는 공개 채널을 쓴다 (절 1.2). 가

격이 매겨지는 방식도, 마켓을 식별하는 문자열의 생김새도 다르다. 이 차이만 보고 두 거래소를 아예 별개의 데이터 계열로 취급하면, 나중에 세 번째 예측시장이 들어올 때마다 스키마를 통째로 새로 설계해야 한다.

그래서 무엇을 바꿨나. 이 시스템은 Kalshi 와 Polymarket 을 같은 제품군으로 본다. 둘 다 이벤트 계약 *(event contract)* 이다. "BTC 가 특정 시각에 특정 가격을 넘는가" 같은 질문에 대해 0 에서 1 사이의 확률처럼 가격이 매겨지고, 사건이 실제로 일어나거나 일어나지 않으면 그 계약은 1 또는 0 으로 정산 (settle) 된다.

왜 그 관점이 문제를 푸는가. 제품군이 같다는 것을 계약 (스키마) 수준에서 인정하면, 두 거래소는 같은 테이블, 같은 필드 집합을 공유하고 거래소를 구분하는 값 하나 (`venue`) 만 갈라지면 된다. 반대로 제품군이 다르다고 보면 거래소마다 테이블을 하나씩 새로 파야 하고, 그 테이블들은 시간이 지나면서 조금씩 다른 모양으로 벌어진다. 이 책에서 계약이라는 말이 반복해서 등장하는데 (장 3), 그 계약의 첫 번째 근거가 바로 이 관점이다.

예제 1.1 (가족과 축) 이 파이프라인은 데이터를 두 가족 *(family)* 으로 나눈다. *pred_*(예측시장: Kalshi, Polymarket)* 와 *crypto_*(현물: Binance, Coinbase)* 다. *venue* 는 가족을 나누는 값이 아니라, 한 가족 안에서 어느 거래소인지를 가리키는 축이다. 예를 들어 *pred_orderbook_ws* 테이블 하나에 *venue=kalshi* 행과 *venue=polymarket* 행이 함께 담긴다. 가족을 가르는 것은 테이블 이름이고, 거래소를 가르는 것은 *venue* 컬럼이다.

1.2 두 거래소, 다른 배선

In brief.

Why " 같은 제품군" 이라는 말이" 코드가 하나로 통합된다" 는 뜻은 아니다. 배선 (연결·인증·구독 단위) 은 거래소마다 다르고, 그 차이를 흡수하는 자리가 따로 있어야 한다.

When producer 가 거래소 WS(웹소켓) 에 연결하는 순간부터 원본 메시지를 파싱해 내보내는 순간까지 적용된다.

Where Kalshi 는 이미 붙어 있는 거래소이고, Polymarket 은 이 시스템에서 스키마 자리만 마련해 둔 상태다 (절 1.4).

How 마켓 식별자, 인증, 구독 단위, 하트비트, 메시지 모양 네 가지 축으로 비교한다.

Kalshi 와 Polymarket 이 계약 수준에서 같은 가족이라는 사실은 절 1.1 에서 확인했다. 이 절은 그 반대편, 즉 두 거래소가 실제로 얼마나 다르게 말을 거는지를 짚는다. 이 차이를 아는 것이 중요한 이유는, 이 차이가" 재구성기 (오더북을 다시 조립하는 로직) 는 거래소 이름을 몰라야 한다" 는 설계 원칙을 낳기 때문이다 (장 2).

축	Kalshi	Polymarket
마켓 식별자	시리즈 형태 문자열 (예: `KXBTCD-26SEP22-B112000`)	CLOB(Central Limit Order Book) token id. 수십 자리 10 진수 문자열
접속	인증이 필요한 WS 채널	공개·무인증 마켓 채널 (`wss://ws-subscriptions-clob.polymarket.com/ws/market`)
구독 단위	시리즈/티커 채널 구독	`clobTokenIds` 목록으로 구독
하트비트	(구현별)	서버가 5 초마다 ping, 클라이언트는 10 초 내 pong 필요
오더북 메시지	`orderbook_snapshot` / `orderbook_delta` 채널	`Orderbook`(스냅샷) / `PriceChange`(델타) / `LastTrade`(체결) 이벤트

Table 1.1: Kalshi 와 Polymarket 의 배선 차이. 계약 (스키마) 은 같지만 여기는 다르다.

티커 모양이 왜 중요한가. Kalshi 의 티커는 사람이 읽을 수 있는 시리즈 문자열이고, Polymarket 의 티커는 숫자로만 이루어진 매우 긴 문자열이다. 이 차이는 미감의 문제가 아니다. 두 거래소의 티커가 우연히 같은 문자열이 되는 일은 실무적으로 없다고 봐도 되지만, 스키마가 그 가능성을 아예 차단하려면 거래소를 구분하는 값이 별도로 필요하다. 장 3 와 장 5 가 이 문제를 다루는 방식 (정렬 키 선두에 `venue` 를 두는 것) 이 여기서 예고된다.

시퀀스 번호가 왜 다르게 취급되는가. Kalshi 는 오더북 갱신마다 단조 증가하는 시퀀스 번호 (`seq`) 를 준다고 가정할 수 있지만, Polymarket 이 같은 방식의 시퀀스를 준다는 보장은 없다. 이 시스템은 그 불확실성을 타입으로 드러낸다. 거래소가 준 원본 시퀀스는 없을 수 있는 값 (`null` 허용) 으로 두고, 재구성기가 필요하면 별도의 값을 합성한다. " 거래소가 보장하는 것" 과" 우리가 만들어 채우는 것" 을 한 필드에 섞지 않는 이유가 여기 있다.

1.3 왜 수집이 어려운가: 델타, 재생 불가, 시계 불일치

무슨 문제가 있었나 (1): 오더북은 델타로 온다. 거래소는" 지금 오더북 전체가 이렇습니다" 라는 스냅샷 (snapshot) 을 매번 보내지 않는다. 대신" 이 가격대의 물

량이 이렇게 바뀌었습니다” 라는 델타 *(delta,* 변화분*)* 를 흘려보낸다. 오더북의 실제 상태를 알려면 최초 스냅샷 위에 그 뒤의 델타를 순서대로 계속 적용해야 한다. 델타 하나를 놓치면, 그 이후로 쌓은 상태 전체가 거래소의 실제 상태와 어긋난다.

그래서 무엇을 바꿨나. 이 시스템은 델타를 받아 적용하는 재구성 *(reconstruction)* 책임을 WS 연결을 쥔 producer 프로세스 안에 둔다. 시퀀스 갭 (간격) 이 감지되면 그 프로세스가 직접 재연결해 새 스냅샷을 받는다.

왜 그 위치가 문제를 푸는가. 갭을 감지하는 것과 갭을 복구하는 것은 다른 일이다. 복구하려면” 다시 스냅샷을 달라” 고 말할 수 있는 권한, 즉 WS 소켓 자체가 필요하다. 재구성 로직을 producer 밖 (예: 별도 컨슈머) 으로 빼면, 갭은 감지할 수 있어도 복구할 방법이 없는 반쪽짜리 설계가 된다. 그래서 재구성은 소켓을 쥔 쪽에 남아야 한다. 이 결론은 장 2 에서”Redis 오더북 라이터는 재구성하지 않는다” 는 설계로 다시 등장한다.

무슨 문제가 있었나 (2): 피드에는 재생이 없다. 거래소 WS 는”5 초 전에 보낸 그 메시지를 다시 보내 달라” 는 요청을 받아 주지 않는다. 재생 *(replay)* 이 없다는 말은 이 뜻이다. 한 번 놓친 메시지는 그 순간의 호가·체결 정보를 영원히 잃는 것과 같다. 이 사실 하나가 장 5 에서 다루는” 메시지가 사라지는 구간을 정확히 알아야 한다” 는 목표의 출발점이다.

무슨 문제가 있었나 (3): 거래소마다 시계가 다르다. ” 이 이벤트가 언제 일어났는가” 라는 질문에 모든 거래소가 같은 방식으로 답하지 않는다. Binance 와 Coinbase 는 메시지 안에 거래소가 매긴 이벤트 시각을 담아 보낸다. 반면 Kalshi 의 틱과 오더북 메시지에는 이 시스템이 신뢰할 수 있는 이벤트 시각 필드가 없다고 알려져 있어서, 이 시스템은 Kalshi 의 해당 경로에 한해 메시지를 받은 시각 *(*수신 시각*)* 을 대신 쓴다. 이것은 임시방편이 아니라 기록된 예외로 남겨 두고, 나중에 이벤트 시각 필드가 확인되면 그쪽으로 옮기기로 정해 둔 상태다.

왜” 시계가 다르다” 는 사실을 숨기면 안 되는가

한 거래소 안에서 어떤 메시지는 거래소 시각을, 어떤 메시지는 수신 시각을 섞어 쓰면, 나중에 이력을 봐도 어느 행이 어느 시계를 따르는지 구분할 방법이 없다. 그래서 이 시스템의 규칙은” 정확한 시각을 항상 써라” 가 아니라” 한 거래소 안에서는 한 시계만 써라, 그리고 어느 시계를 쓰는지 기록해 두라” 이다. 정확도보다 일관성과 투명성을 우선한 것이다.

1.4 BTC 한 종목으로 시작하는 이유

무슨 문제가 있었나. Kalshi 한 거래소만 해도 취급하는 시리즈가 암호화폐, 날씨, 정치 등으로 넓게 퍼져 있고, 현물 거래소도 여러 심볼을 동시에 스트리밍할 수 있다. 처음부터 전 종목을 받으면, 파이프라인이 제대로 동작하는지 확인하기도 전에 메시지량과 실패 지점의 수가 함께 불어난다.

그래서 무엇을 바꿨나. 이 시스템은 BTC 한 종목으로 구독 범위를 좁힌다. Binance 는 `BTCUSDT`, Coinbase 는 `BTC-USD`, Kalshi 는 BTC 관련 시리즈만 남긴다.

왜 그 변경이 문제를 푸는가. 이 제한은 스키마가 아니라 구독 설정에 건다. 즉 테이블이나 proto 메시지에"BTC 만 허용" 이라는 조건이 박혀 있는 것이 아니라, producer 가 거래소에 무엇을 구독할지 정하는 상수 하나가 좁혀져 있을 뿐이다. 그래서 다른 종목을 추가하는 일은 스키마 변경이 아니라 설정 변경이 되고, 이는 나중에 env 값 하나로 뺄 수 있는 1 줄짜리 과제로 남는다.

1.5 시스템의 목적: " 적어도 한 번" 에서" 정확히 한 번" 으로

앞의 세 절이 확인한 어려움. 델타로만 오는 오더북, 재생이 없는 피드, 거래소마다 다른 시계. 은 전부" 메시지 하나를 잃어버리거나 두 번 받을 수 있다" 는 사실로 수렴한다. 이 시스템이 스스로에게 거는 목표는 그 위험을 완전히 없애는 것이 아니다. 완전한 제거는 스트리밍 시스템에서 현실적인 목표가 아니다. 대신 이 목표를 *at-least-once(*적어도 한 번 전달*)* 에서 *effectively-exactly-once(*사실상 정확히 한 번*)* 로 가는 길로 잡는다. 배달은 유실이 없거나 (또는 유실이 어디서 나는지 정확히 알려져 있고), 중복은 저장 단계에서 안전하게 접힌다는 뜻이다. 이 목표가 정확히 무엇을 뜻하는지, 그리고 이 파이프라인의 어느 구간이 그 목표를 실제로 만족하는지는 장 5 가 전담해서 다룬다. 지금 독자가 들고 가야 할 것은 한 문장이다. *" 메시지가 사라지는 구간이 있다면, 그 구간이 어디인지는 반드시 알 수 있어야 한다."*

이어지는 장 2 는" 그 구간을 어디에 둘 것인가" 라는 질문에서 출발해, Kafka 와 Redis 와 ClickHouse 가 왜 지금의 역할로 나뉘어 있는지를 다룬다.

Chapter 2

왜 Kafka 인가, 왜 Redis 와 ClickHouse 둘 다인가

In brief.

Why ” 현재 오더북이 어떤 상태인가” 와” 지금까지 무슨 일이 있었는가” 는 같은 질문이 아니다. 이 장은 그 두 질문을 하나의 저장소로 뭉뚱그리면 왜 무너지는지, 그래서 왜 Redis(현재 상태) 와 ClickHouse(이력) 가 함께 필요한지를 다룬다.

When 세 producer 가 거래소 WS 를 구독하기 시작한 순간부터, 대시보드가 값을 읽어 그리는 순간까지 전체 토폴로지에 적용된다.

Where Kalshi·Polymarket·Binance·Coinbase 데이터 전부가 같은 토폴로지를 지난다 (장 1 에서 본 대로 계약은 같은 가족 안에서 공유된다).

How 먼저”Redis 만으로 될까” 라는 실패하는 설계를 확인하고, 그 실패가 요구하는 역할 분리를 정리한 다음, Kafka 가 그 사이에서 하는 일과 mock 위에서 이 전체 흐름을 먼저 증명하는 이유를 본다.

2.1 먼저 실패하는 설계: 전부 Redis 에 담으면 어떻게 되는가

무슨 문제가 있었나. 오더북 라이터를 설계할 때 세 가지 안이 있었다. 그중 하나는 producer 가 Redis Streams 에 델타를 직접 써넣고, 라이터가 그 스트림을 소비하며 스스로 오더북을 재구성하는 안이었다. 이 안은 Kafka 를 완전히 우회한다. Kafka 에 쓰지 않고 곧장 Redis 에 쓰기 때문에, 배선이 한 단계 짧아진다는 장점처럼 보인다.

그래서 무엇이 기각됐나. 이 안은 채택되지 않았다. 이유는 두 가지다. 첫째, Redis Streams 를 우회 경로로 쓰는 순간" 모든 데이터는 계약을 거쳐 저장된다" 는 불변식이 깨진다. 이 시스템에서 Kafka 는 단순한 전송 수단이 아니라 장 3 가 정의하는 계약이 지나가는 유일한 관문이다. 둘째, 이 안은 아직 검증되지 않은 길이다. producer 가 델타를 직접 밀어 넣는 경로, 라이터가 그 델타로 상태를 재구성하는 로직 둘 다 이 시스템에서 한 번도 운영해 본 적이 없는 조합이다. 판정은 명확히" 기각 (YAGNI. 아직 필요하지 않은 것을 먼저 만들지 않는다)" 이다.

왜 이 실패가 역할 분리를 가리키는가. 이 안이 기각된 이유를 뒤집어 보면 설계 원칙이 나온다. "Redis 에만 있고 다른 곳에는 없는 데이터를 만들지 않는다" 는 원칙이다. Redis Streams 안으로만 들어가고 마는 델타는 Redis 가 통째로 사라지면 함께 사라진다. 그런데 Redis 는 원래 캐시 *(cache)* 역할을 맡기기로 되어 있었다. 그리고 캐시는 사라져도 괜찮아야 캐시다. 사라지면 안 되는 데이터를 캐시 안에만 두는 설계는 그 자체로 모순이다. 이 모순이 절 2.2 의 역할 분리로 이어진다.

2.2 Redis 는 현재 상태, ClickHouse 는 진실

In brief.

Why " 지금 오더북이 어떤 모양인가" 에 빠르게 답하는 일과, " 지난 한 시간 동안 무슨 일이 있었는가" 에 정확하게 답하는 일은 요구하는 저장소의 성질이 다르다.

When Redis 는 대시보드가 호가창을 그릴 때, ClickHouse 는 이력·차트·집계를 조회할 때 각각 읽힌다.

Where 이 구분은 예측시장·현물 데이터 모두에 동일하게 적용된다.

How 한쪽은" 사라져도 다음 메시지가 오면 복구된다", 다른 쪽은" 사라지면 재현할 방법이 없다" 는 성질로 역할을 가른다.

무슨 문제가 있었나. 절 2.1 에서 확인했듯, 한 저장소에" 현재 상태" 와" 전체 이력" 을 함께 담으면 그 저장소가 요구받는 성질이 서로 충돌한다. 현재 상태는 최신 값 하나만 빠르게 읽히면 충분하지만, 이력은 과거의 모든 값이 하나도 빠짐없이 남아 있어야 한다. 하나의 저장소로 둘 다 잘하려 하면, 어느 한 쪽을" 일단 되는 수준" 으로 타협하게 된다.

그래서 무엇을 바꿨나. 이 시스템은 저장소를 둘로 가른다. Redis 는 티커당 최신 스냅샷 한 개와 라이터의 하트비트만 담는다. ClickHouse 는 모든 스냅샷·델타·틱·체결의 전체 이력을 담는다.

	Redis	ClickHouse
담는 것	티커당 최신 스냅샷 1 개, 라이터 하트비트	모든 스냅샷/델타/틱/체결의 전체 이력
접근	`apps/api` 가 수 ms 단위로 읽음	`apps/api` 가 차트/카운트용으로 읽음
유실 시	다음 메시지에서 자동 복구됨. 진실 *(truth)* 이 아니다	재현 불가능. 진실이다
쓰기 주체	`orderbook-writer` 단독	ClickHouse Kafka 엔진 (MVP2 이후)

Table 2.1: Redis(현재 상태) 와 ClickHouse(이력) 의 역할 분리.

왜 그 변경이 문제를 푸는가. 이 표의 핵심 문장은" 유실 시" 행이다. *Redis* 가 통째로 날아가도 복구 절차가 필요 없어야 한다. 다음 메시지가 라이터에 도착하면 그 메시지 하나로 최신 상태가 다시 채워지기 때문이다. 이 성질을 지키는 한 Redis 는 진짜 의미의 캐시로 남는다. 반대로 ClickHouse 가 잃는 행은 영영 사라진다. 거래소 피드에 재생이 없다는 사실을 절 1.3 에서 이미 확인했다. 그래서 ClickHouse 가 진실이라는 이름을 갖는다. 이 두 성질은 설계 규칙 하나로 요약된다. 이 성질을 깨는 설계 *(Redis* 에만 있는 데이터*)* 를 넣지 않는다.

대시보드는 왜 두 저장소를 나눠 읽는가. " 현재 오더북" 은 Redis 에서 읽는다. 이 값은 최신 상태 하나만 있으면 되고, 수 ms 안에 응답해야 화면이 매끄럽다. 반면 차트나 집계, " 얼마나 오래됐는가" 같은 수집 현황은 ClickHouse 에서 읽는다. 이 구분은 단순한 성능 최적화가 아니다. Redis 에 없고 ClickHouse 이력에만 있는 티커를" 현재 오더북 있음" 으로 잘못 응답하면, 그 자체가 정확성 문제가 된다. 그래서" 현재 오더북" 라우트는 ClickHouse 로 폴백 (fallback) 하지 않는다. 이 라우트가 실제로 ClickHouse 를 읽기 시작하는 것은 MVP2 이후, 수집 카운트·freshness(신선도) 용도의 별도 엔드포인트가 생길 때다.

병합이 끝나기 전의 중복은 어느 쪽에 영향을 주는가

ClickHouse 의 중복 제거는 즉시 일어나지 않는다 (장 5). 이 지연이" 현재 오더북" 에는 영향을 주지 않는 이유가 바로 이 절의 역할 분리다. 현재 오더북은 애초에 ClickHouse 가 아니라 Redis 에서 읽으므로, ClickHouse 쪽 병합 지연의 영향권 밖에 있다.

2.3 Kafka 가 그 사이에 있는 이유

무슨 문제가 있었나. Redis 와 ClickHouse 는 서로 다른 두 소비자다. 하나는" 현재만", 하나는" 전부" 를 원한다. producer 는 셋 (Kalshi, Binance, Coinbase) 이

고, 소비자도 여럿 (Redis 오더북 라이터, ClickHouse Kafka 엔진, 이후 늘어날 수 있는 다른 소비자) 이다. producer 가 각 소비자에게 직접 데이터를 쏘는 구조를 그리면, producer 수 × 소비자 수만큼의 배선이 생기고 producer 하나가 소비자 하나의 장애에 발이 묶일 위험이 생긴다.

그래서 무엇을 바꿨나. 세 producer 는 전부 원격 Kafka 하나로 모이고, 그 뒤에서 Redis 오더북 라이터와 ClickHouse Kafka 엔진이 각자 독립적으로 같은 토픽을 읽는다.

왜 그 위치가 문제를 푸는가. Kafka 가 producer 와 소비자 사이에 서면, producer 는" 누가 이 데이터를 읽는가" 를 몰라도 된다. 그저 정해진 토픽에 계약이 정한 형태로 내보내기만 하면 된다. 소비자 쪽도 마찬가지다. Redis 라이터가 다운되어 있어도 ClickHouse 쪽 적재는 영향을 받지 않는다. 이 독립성이 절 2.2 의 역할 분리를 실제로 가능하게 만드는 배선이다. 두 소비자가 같은 지점 (Kafka 토픽) 에서 갈라지기 때문에, 한쪽 저장소의 설계를 바꿔도 다른 쪽 producer 나 소비자를 건드릴 필요가 없다.

producer 한 곳 (Kalshi WS 를 구독하는 프로세스) 이 오더북 델타를 스냅샷으로 재구성한 뒤, 그 결과를 두 개의 토픽으로 나눠 낸다는 점도 이 자리에서 짚어 둘 만하다. 원본 델타 (감사·재현용) 와 적용 후 전체 북 (소비자용) 이다. 재구성 자체는 절 1.3 에서 본 이유로 producer 안에 남아야 하고, Redis 라이터는 그 결과물 (전체 북) 만 받아 그대로 저장하는 무상태 *(stateless)* 컴포넌트로 설계된다. 재구성 로직이 라이터 안에도 한 벌 더 생기면, 두 재구성기가 서로 다른 결론에 도달해 발산 (divergence) 할 위험이 생기기 때문이다.

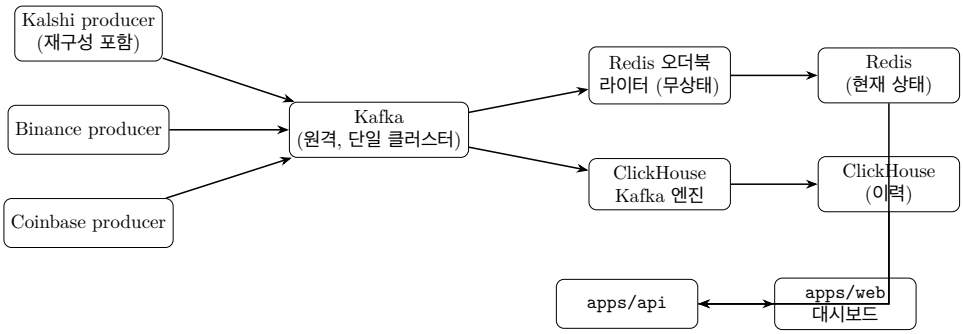


Figure 2.1: 세 producer 가 Kafka 하나로 모이고, 두 독립 소비자 (Redis 라이터, ClickHouse Kafka 엔진) 가 갈라진다. `apps/api` 는 Redis 에서 현재 상태를, ClickHouse 에서 이력을 각각 읽어 대시보드에 낸다.

2.4 왜 흐름을 먼저 mock 으로 증명하는가

무슨 문제가 있었나. 위 토폴로지를 그림으로 그리는 것과, 그 토폴로지가 실제로 끝까지 도는 것을 확인하는 것은 다른 일이다. Kafka 연결, producer 인증, 실 거래소 피드까지 한 번에 준비해야 이 흐름을 눈으로 볼 수 있다면, 배선 자체가 맞는지 틀린지도 그 준비가 끝나야 알 수 있다. 이 저장소가 따르는 단계별 구축 원칙 (MVP 래더) 은 이 순서를 뒤집는 것을 금지한다. 아무도 실행해 본 적 없는 흐름을 위한 배관을 먼저 놓지 않는다는 원칙이다 (‘.claude/CLAUDE.md‘ㄱSystem build phasing˩).

그래서 무엇을 바꿨나. 이 시스템의 첫 단계는 Kafka 연결이 *0* 개인 채로 시작한다. Redis 오더북 라이터는 `/mocks/*.jsonl` 파일을 실시간처럼 재생하는 `mockSource` 를 입력으로 받고, `apps/api` 와 `apps/web` 은 처음부터 실물 코드로 이 mock 데이터를 읽어 화면까지 그린다.

왜 그 변경이 문제를 푸는가. 핵심은 `apps/api` 가 처음부터 바뀌지 않는다는 사실이다. 이 프로세스는 Redis 와 ClickHouse 만 읽고 Kafka 를 직접 알지 못한다. 그래서 mock 단계의 `apps/api` 코드와, 나중에 실 Kafka 가 붙은 단계의 `apps/api` 코드는 바이트 단위로 같다. mock 을 위한 API 를 별도로 만드는 대신, mock 의 경계를 API 보다 한 층 아래 (라이터가 데이터를 어디서 읽어오는가) 로 내린 것이다. 이 설계 덕분에” 드롭인 (drop-in) 교체 지점” 이 정확히 두 곳으로 좁혀진다. 라이터의 입력을 정의하는 `MessageSource` 인터페이스, 그리고 ClickHouse 적재 경로다. 이 둘을 실 구현으로 바꿔 끼우는 것만으로 이후 단계가 성립하고, 나머지 코드는 손대지 않는다.

계층	1 단계 (mock)	2 단계 (현물 1 종 실 연동)	3 단계 (예측시장 실 전환)
`apps/web`	변화 없음	변화 없음	변화 없음
`apps/api`	변화 없음	변화 없음	변화 없음
Redis 내용	`mockSource`	`mockSource` 그대로	실 라이터 + `kafkaSource`
ClickHouse 내용	mock 데이터 직접 삽입	현물 2 개만 Kafka 엔진 + MV	예측시장 4 개도 Kafka 엔진 + MV

Table 2.2: mock 경계는 API 아래로 내려가 있다. 드롭인 지점은 `MessageSource` 구현과 ClickHouse 적재 경로 두 곳뿐이다.

mock 하나만으로는 부족한 이유. mock 데이터셋에는 Kalshi 행만이 아니라 Polymarket 행도 최소 하나 반드시 들어간다. 이유는 강제 장치다. Kalshi 하나만

mock 에 넣으면, `apps/api` 와 `apps/web` 이" 티커는 항상 `KX` 로 시작한다" 같은 가정을 코드에 몰래 박아 넣어도 타입체크와 화면 모두 통과해 버린다. Polymarket 의 수십 자리 숫자 티커를 처음부터 섞어 두면, 그런 가정은 mock 단계에서 곧바로 깨진다. 나중에 실제 Polymarket 연동이 붙는 단계가 아니라 지금 깨지는 편이 싸다.

운영 안전장치. mock 구현은 명시적인 스위치 (`MOCK_SOURCE=1`) 를 켜야만 선택되고, 기본값은 실 데이터를 읽는 쪽이다. 이 순서를 뒤집어 mock 을 기본값으로 두면, 운영 환경에서 mock 이 조용히 도는 사고로 이어질 수 있다. 스위치는" 켜야 mock" 이어야 하지" 꺼야 실데이터" 여서는 안 된다.

2.5 정리: 이 장이 다음 장으로 넘기는 것

이 장에서 확인한 것은 세 가지다. 하나, Redis 와 ClickHouse 는" 현재" 와" 전부" 라는 서로 다른 요구를 각각 맡는다. 둘, Kafka 는 producer 와 두 독립 소비자 사이에서 배선의 수를 줄이고 서로의 장애를 격리한다. 셋, 이 토폴로지는 Kafka 연결이 0 개인 mock 위에서 먼저 흐름 전체를 증명한 뒤에야 실 연동으로 넘어간다. 이 세 가지가 성립하려면 producer, Kafka, Redis, ClickHouse 가 같은 형태의 데이터를 주고받는다는 전제가 필요하다. 그 전제를 지키는 장치가 장 3 가 다루는 스키마 계약이다.

Chapter 3

스키마를 계약으로 두는 설계

In brief.

Why 메시지의 모양을 정의하는 곳이 여러 곳으로 흩어지면, 그중 하나만 고치고 나머지를 잊는 실수가 언젠가 반드시 일어난다. 이 장은 그 실수를 구조적으로 막는 설계, 즉 하나의 계약을 여러 층이 그대로 반복하는 사슬을 다룬다.

When proto 메시지를 정의하는 순간부터, 그 값이 Kafka 를 거쳐 ClickHouse 테이블에 행으로 앉는 순간까지 전 구간에 적용된다.

Where 예측시장 (`pred_*`) 과 현물 (`crypto_*`) 양쪽 스키마 모두 같은 원칙을 따른다.

How proto 가 형태 (shape) 의 단일 원본이고, codec 이 그 형태를 바이트로 옮기며, ClickHouse DDL 이 그 형태를 저장 형태로 받는다. 세 층의 순서를 지키면 계약은 하나로 유지된다.

3.1 계약이 세 층인 이유

무슨 문제가 있었나. 데이터의" 모양" 을 정의하는 자리가 여럿이면, 그 정의는 반드시 서로 어긋난다. 이 저장소가 물려받은 구 설계는 계약을 네 단계로 나눠 두고 있었는데, 그중 두 층 (`table-configs/`, `generators/`) 은 예전 저장소 (TimescaleDB 시절) 의 유물이었다. `hypertable`, `TIMESTAMPTZ`, `NUMERIC` 같은 개념이 현재 쓰는 ClickHouse DDL(컬럼이 전부 `String`) 과 이미 맞지 않았다. 즉" 정의하는 자리" 라고 적혀 있었지만 실제로는 아무도 참조하지 않는 죽은 문서였다.

그래서 무엇을 바꿨나. 계약 체인을 세 층으로 정리한다.

1. `packages/schema/src/proto/*.proto`. 형태의 단일 원본 (SSOT, single source of truth)
2. `packages/schema/src/proto-codec.ts`. 인코딩/디코딩. 와이어 포맷은 JSON(절 3.2)
3. `packages/db/migrations/clickhouse/*.sql`. 저장 형태

변경은 반드시 이 순서로 흐른다. 타입 정의를 다른 곳에 복제하지 않는다.

왜 그 순서가 문제를 푸는가. 세 층이 같은 계약을 다른 표현으로 반복하기 때문에, 필드 하나를 추가하는 변경은 세 층 모두에서 같은 이름으로 나타난다. 1 번 층이 형태를 정하고, 2 번 층은 그 형태를 손대지 않고 바이트로 옮기는 방법만 바꾸며, 3 번 층은 같은 형태를 저장할 컬럼으로 받는다. 이 순서를 지키면, " 제안된 변경이 계약과 맞는가" 라는 질문에 항상 하나의 답만 존재한다. proto 가 그 형태를 이미 허용하는지 보면 된다. 계약이 여럿이었다면 이 질문에 층마다 다른 답이 나올 수 있었다.

이 장 전체를 관통하는 예제가 `venue` 필드 하나를 추가한 변경이다 (절 3.4). 이 변경이 세 층을 순서대로 통과하고 나서, 병렬 테이블도 별도 마이그레이션도 없이 끝났다는 사실이" 계약이 하나여야 하는 이유" 를 가장 구체적으로 보여 준다.

3.2 와이어 포맷이 JSON 인 이유, 그리고 이것이 잠정 결정인 이유

무슨 문제가 있었나. proto 바이너리는 스키마 없이는 한 글자도 읽을 수 없다. 콘솔 컨슈머로 토픽을 열어 페이로드를 눈으로 확인하려 해도, 바이너리 앞에서는 아무것도 보이지 않는다. 파이프라인이 아직 검증되지 않은 이 단계에서, " 지금 토픽에 무엇이 쌓이고 있는지" 눈으로 확인하지 못하는 것은 디버깅을 막는 직접적인 장벽이다.

그래서 무엇을 바꿨나. MVP1 부터, 그리고 데이터 파이프라인이 잘 도는 것이 확인될 때까지 Kafka 와이어 포맷은 JSON 이다. proto 메시지의 형태는 그대로 proto 가 원본으로 남고, 바뀌는 것은 그 형태를 바이트로 옮기는 방법뿐이다.

왜 그 변경이 문제를 푸는가. `@bufbuild/protobuf` 라이브러리가 `toJson`/`fromJson` 을 제공하므로, 코덱 경계에서 바뀌는 것은 방향당 함수 하나뿐이다. 스키마 마이그레이션은 일어나지 않는다. 되돌릴 때도 마찬가지다. 복귀 경로는 `toJsonBytes`/`fromJsonBytes` 안에서 `toJson` 을 `toBinary` 로 바꾸는 코드 한 줄이지, 환경변수로 두 포맷을 동시에 지원하는 런타임 스위치가 아니다. 스위치를 남기면"JSON 바이트에 `encoding: 'proto'` 헤더만 붙는" 거짓 상태가 생길 수 있으므로, 이 시스템은 한 시점에 하나의 와이어 포맷만 존재한다는 규칙을 지킨다.

이것은 영구 결정이 아니다
이 결정에는 조건이 붙어 있다. ” 파이프라인이 잘 도는 것이 확인될 때까지” 다. 종료 조건은 실 Kalshi 오더북이 Redis 에 차고 `FINAL` 조회가 중복 0 을 반환하는 단계다. 그 시점에 바이너리 복귀를 재검토한다. 자동으로 되돌아가는 것이 아니라, 재검토가 필요한 잠정 결정이라는 뜻이다.

변경 범위가 왜 두 곳으로 묶이는가. 이 결정이 건드리는 자리는 코덱 경계 (`proto-codec.ts` 의 인코더/디코더, 라우팅의 인코딩 분기) 와 ClickHouse Kafka 엔진의 포맷 문자열 (MVP2) 두 곳뿐이다. 테이블 DDL, 정렬 키, idempotency key(멱등성 키) 함수, Redis 키 형태는 손대지 않는다. 와이어 포맷은 저장 형태가 아니다. 이 한 문장이 변경 범위를 좁게 유지하는 근거다.

mock 데이터가 이 결정의 가장 강한 증거를 만든다. 와이어가 JSON 이 되면서, `/mocks/*.jsonl` 의 한 줄은 producer 가 Kafka 에 실제로 보낼 바이트와 글자 그대로 동일해진다. 이전에는” 필드 단위로 비슷하다” 는 정도의 유사성이었는데, 이제는 완전한 동일성이다. mock 줄을 그대로 콘솔 프로듀서에 부어도 성립하고, 실 토픽에서 퍼낸 줄을 그대로 mock 파일에 붙여도 성립한다. 계약의 참조 구현이 사람이 읽고 손으로 옮길 수 있는 물건이 된 것이다. 이것이 JSON 전환의 가장 강한 논거다.

3.3 규칙이 조용히 깨지는 세 지점

무슨 문제가 있었나. JSON 은 유연한 포맷이라, 코덱이 기본값 그대로 동작하게 두면 그 기본값이 조용히 계약을 어긴다. 세 가지 기본값이 특히 위험하다.

규칙 1: 필드명은 proto 이름을 그대로 쓴다. protobuf-JSON 표준의 기본값은 lowerCamelCase(예: `timeMs`) 다. 그런데 ClickHouse 컬럼은 snake_case (`time_ms`) 다. 쓰기 쪽에서 `useProtoFieldName: true` 옵션을 빠뜨리면, 필드명 불일치가 조용한 데이터 손실로 이어진다. 실측 결과, `timeMs`(camelCase) 키를 `time_ms` 컬럼에 `JSONEachRow` 로 넣으면 서버 기본 설정에서 *INSERT* 가 *exit 0* 으로 성공하면서 컬럼이 조용히 *0* 이 된다. 에러도 경고도 없다.

규칙 2: int64 는 따옴표 친 문자열로 나간다. JS 의 `number` 는 2^{53} 을 넘는 정수를 정확히 담지 못하므로, protobuf-JSON 표준은 int64 계열을 JSON 문자열로 직렬화한다. 이것은 끌 수 있는 옵션이 아니라 표준 자체다. 실측으로 확인한 바, ClickHouse 의 `JSONEachRow` 는 따옴표 친`"1758500000123"` 을 추가 설정 없이 `Int64` 컬럼과 `DateTime64(3)` 컬럼 양쪽에 정확히 받아들인다.

규칙 4: 기존 JSON 경로를 그대로 재사용하면 안 된다. 이 저장소가 물려받은 `publishJson()` 함수는 페이로드에 `_ts`, `_key` 를 주입한다. 이 두 키는 proto 스키마에 없는 값이다. 여기서 함정이 하나 갈라진다. " 규칙 4 가 와이어 페이로드를 규율한다" 는 것을" 인코더에 `_key` 를 아예 넘기지 말라" 로 잘못 읽으면, 인코더가 `idempotency_key` 를 채울 재료를 잃는다. 그 결과 모든 행의 *idempotency_key* 가 빈 문자열이 되고, 이는 정렬 키의 마지막 성분이므로 (장 5) 중복 제거가 아무 에러 없이 통째로 무력화된다. 올바른 형태는 라우터가 인코더에 `{...data, _key: key}` 를 입력으로 넘기고, `toJson` 이 스키마에 선언된 필드만 방출해 `_key` 가 와이어에는 실리지 않는 것이다. 규칙이 규율하는 대상 (와이어) 과 규율하지 않는 대상 (인코더 입력) 을 혼동하지 않는 것이 이 규칙의 핵심이다.

3.4 한 단어가 세 가지 뜻을 지고 있었다: `venue`, `ingest_path`, `feed`

무슨 문제가 있었나. Polymarket 을 두 번째 예측시장 거래소로 들이기로 하면서, 예측시장 메시지에" 어느 거래소인가" 를 담는 필드가 필요해졌다. 그런데 이 개념을 표현할 후보 단어 `source` 는 이미 이 저장소 안에서 세 가지 서로 다른 뜻으로 쓰이고 있었다.

- 현물 (crypto) 메시지의 `source` 는 `binance`/`coinbase` 값을 가진다. 이것은 이미" 어느 거래소인가" 라는 뜻이다.
- Kalshi 계열 메시지의 `source` 는 `live_ws`/`live_rest` 값을 가진다. 이것은" 어느 수집 경로로 받았는가" 라는 다른 뜻이다.
- API 응답의 `source` 는 `mock`/`live_ws` 값을 가진다. 이것은"mock 인가 실데이터인가" 라는 또 다른 뜻이다.

한 단어가 세 자리에서 다른 것을 가리키고 있었으므로, 코드를 읽는 사람은 특정 `source` 값을 볼 때마다" 이게 지금 거래소를 말하는지, 경로를 말하는지, mock 여부를 말하는지" 를 문맥으로 추측해야 했다.

그래서 무엇을 바꿨나. 세 개념에 세 이름을 각각 준다.

현물 메시지의 `source` 는 이미 venue 개념이었으므로 필드를 추가하지 않고 이름만 `venue` 로 바꾼다. 필드 번호가 그대로라서 와이어 호환이 유지된다. Kalshi 계열의 `source` 는 다른 개념이었으므로 `ingest_path` 로 개명한다. 예측시장 메시지에는 `venue` 필드를 신규로 추가한다.

개념	이름	값
데이터를 만든 거래소/운영주체	`venue`	`kalshi`, `polymarket`, `binance`, `coinbase`
수집 경로 (WS/REST)	`ingest_path`	`live_ws`, `live_rest`
mock/실데이터 구분 (API 응답 전용, DB 컬럼 아님)	`feed`	`mock`, `live`

Table 3.1: 한 단어가 지던 세 가지 뜻을 세 이름으로 분리한다.

왜 그 변경이 문제를 푸는가. 이름을 나누면, 코드를 읽는 사람이 값 하나만 보고도 그것이 무엇을 뜻하는지 알 수 있다. 이것이" 계약이 하나여야 한다" 는 이 장의 주제가 필드 이름 수준까지 내려온 모습이다. 한 이름은 한 개념만 가리켜야 값을 보는 쪽과 읽는 쪽이 같은 것을 생각한다.

예제 3.1 (같은 실패가 뷰 모델에서도 반복됐다) 대시보드가 오더북을 렌더링할 때 쓰는 뷰 모델 *(`OrderbookSnapshot`,* 절 *3.7)* 에서도 같은 종류의 문제가 나타났다. *`kalshi-core`* 패키지 안에 *Kalshi WS* 파싱용 내부 타입 하나가 같은 이름 *`OrderbookSnapshot`* 으로 존재하고 있었다. 한 이름이*"API* 가 대시보드에 내는 뷰 모델*"* 과*"Kalshi WS* 메시지를 파싱한 내부 표현*"* 이라는 두 뜻을 동시에 지고 있었던 것이다. 해법은 *`venue/ingest_path/feed`* 를 가른 것과 정확히 같다. 거래소 방언 타입은 거래소 이름을 붙여 *`KalshiWsOrderbook`* 으로 개명하고*, API* 계약의 이름 *`OrderbookSnapshot`* 은 절 *3.7* 가 다루는 뷰 계층에만 남긴다.

3.5 `venue` 가 정렬 키 선두에 서는 이유

무슨 문제가 있었나. ClickHouse 테이블의 정렬 키 (sort key, `ORDER BY`) 는 단순한 정렬 순서가 아니다. ClickHouse 의 스파스 인덱스 (sparse index) 는 정렬 키 접두사부터 효력을 갖고, 장 5 가 다루는 중복 제거도 이 정렬 키가 완전히 같은 행만 하나로 접는다. 그래서 정렬 키에 어떤 컬럼을 얼마나 앞쪽에 두는가는 조회 성능과 중복 제거 양쪽에 함께 영향을 준다.

그래서 무엇을 바꿨나. 이 시스템의 데이터 테이블은 모두 `ORDER BY (venue, ticker, time, idempotency_key)` 형태를 쓴다. `venue` 가 맨 앞이다.

왜 그 순서가 문제를 푸는가. 이 절이 다루는 것은 `venue` 가 선두에 있어야 하는 이유 중 인덱스와 압축 두 가지다 (우연한 티커 충돌을 막는 이유는 예제 5.1 이 이미 다뤘다).

1. 인덱스 효율. `venue` 는 값이 2 4 종뿐인 저카디널리티 (cardinality, 값의 종류 수) 컬럼이다. 저카디널리티 컬럼이 정렬 키 앞에 있으면, 스파스 인덱스가 `WHERE venue='kalshi'` 같은 조건에서 파트 대부분을 인덱스 단계에서부터 걸러낼 수 있다. 정렬 키는 접두사부터 효력을 갖는다는 규칙 때문에, 저카디언리티 컬럼이 `ticker` 뒤에 있으면 이 이득이 사라진다.

2. 압축률. 정렬이 `venue` 별로 먼저 묶이면, `ticker` 문자열이 venue 별로 연속 배치된다. Kalshi 티커 (`KXBTC…` 형태) 와 Polymarket 티커 (긴 10 진수) 는 형태가 완전히 다르므로, 같은 venue 끼리 묶인 상태가 압축기에 더 유리한 패턴을 만든다.

기존 현물 (crypto) 테이블이 이미 `(source, symbol, …)` 형태로 venue 성격의 컬럼을 선두에 두고 있었다는 사실도 이 순서를 뒷받침한다. 예측시장 쪽만 다르게 두면 두 가족 (예제 1.1) 의 정렬 규칙이 갈라진다.

정렬 키를 바꿔도 병합 동작은 손해를 보지 않는다

`venue` 를 선두에 추가해도 ReplacingMergeTree 의 중복 제거 능력은 약해지지 않는다. `venue` 는 메시지마다 결정적으로 정해지는 값이라, 같은 이벤트의 재전송은 여전히 매번 같은 `venue` 값을 갖고, 접히는 행의 집합은 줄지 않는다. 오히려 `idempotency_key`(멱등성 키) 에도 `venue` 가 들어가면서 식별자가 더 강해진다 (장 5).

3.6 와이어 포맷이 바뀌어도 형태의 원본은 proto 다

무슨 문제가 있었나. 절 3.2 에서 와이어 포맷을 JSON 으로 바꾸기로 한 결정이, ” 그럼 이제 스키마의 원본은 JSON 스키마인가” 라는 오해를 부를 수 있다. proto 를 버리고 JSON 형태를 새로 어딘가에 손으로 적으면, 다시 계약이 두 곳으로 갈라지는 실수를 반복하게 된다.

그래서 무엇을 바꿨나. proto 는 그대로 형태의 단일 원본으로 남는다. 바뀐 것은 그 형태를 바이트로 옮기는 함수 하나뿐이다 (절 3.2). 부수 효과로, JSON 에는 `format_schema` 라는 개념 자체가 없으므로, ClickHouse 가 proto 스키마 파일을 컨테이너 안에 마운트할 필요도 사라졌다. 이 마운트는 과거 ClickHouse 엔트리포인트가 해당 경로의 소유권을 바꾸려다 실패해 컨테이너 재시작 루프를 만든 적이 있는 지점이었다. JSON 전환이 일거리를 늘리는 대신 그 문제를 없애며 해결한 셈이다.

왜 이 구분이 중요한가. ” 형태의 원본” 과” 와이어로 나가는 표현” 을 같은 것으로 착각하면, 와이어 포맷이 바뀔 때마다 형태 정의도 함께 옮겨야 한다는 잘못된 결

론에 이른다. 이 두 층을 분리해 둔 덕분에, 절 3.2 의 결정이" 잠정적" 이라고 말할 수 있다. 바이너리로 되돌리는 일이 형태를 다시 정의하는 일이 아니라, 코덱 안의 함수 호출 하나를 바꾸는 일이기 때문이다.

3.7 파생 타입은 파생 원본 옆에 산다

무슨 문제가 있었나. 대시보드가 오더북을 그릴 때 쓰는 응답 형태 (`OrderbookSnapshot`)는 proto 메시지가 아니라, Kafka 에 흐르는 형태에서 파생된 *API* 계약이다. 이 타입이 한때 세 곳에 동시에 있었다. 이 문서의 산문 설명, `apps/api` 안의 정의, 그리고 `apps/web` 안의 정의였다. 뒤의 둘은 한 글자도 다르지 않은 복제였다. 개발자가 import 대신 복제를 택한 데는 그 나름의 근거가 있었다. 이 저장소는 앱과 앱 사이에 직접 의존 간선을 두지 않는 규칙을 지키므로, 두 앱을 억지로 묶어 한쪽을 다른 쪽 배포에 끌고 들어가는 것보다는 복제가 안전해 보였다.

그래서 무엇을 바꿨나. 이 타입을 `packages/schema` 안, `proto/·proto-codec.ts` 와 나란한 `view/` 디렉터리에 둔다. 새 패키지 `packages/types` 는 만들지 않는다.

왜 그 자리가 문제를 푸는가. `OrderbookSnapshot` 은 이미"proto 가 아니라 API 계약이며, `PredOrderbook` 에서 파생된다 (별도 SSOT 아님)" 고 정의돼 있었다. 파생물은 파생 원본 옆에 산다는 것이 판정의 근거다. 별도 패키지를 만들면 그 타입이 독립된 계약처럼 보이게 되어, " 별도 SSOT 가 아니다" 라는 원래 문장과 모순된다. 복제가 위험한 이유도 이것과 같은 축에 있다. `apps/api` 가 필드를 늘리거나 값의 범위를 넓혀도 `apps/web` 의 복제본은 타입체크를 그대로 통과하고, 값이 화면에서 깨질 때에야 어긋남이 드러난다. 파생 관계를 코드가 아니라 기억에 의존해 유지하면, 그 기억은 결국 실패한다.

이름에 관한 판단 하나 더. 이 저장소의 패키지는 전부 도메인 이름을 쓴다. `schema`, `db`, `common`, `kalshi-core`, `env`. `types` 라는 이름은 언어 기능의 이름일 뿐이라, " 이 패키지에 무엇이 들어와도 되는가" 를 스스로 방어할 문장이 없다. 그런 이름의 패키지는 결국 갈 곳을 정하지 못한 타입들의 집합소가 된다. 반면 `schema/view/`는"proto 메시지에서 파생되지 않는 타입은 여기 들어올 자격이 없다" 는 문장을 갖는다. 이 문장 하나가 `view/`가" 타입 일반" 으로 번지는 것을 막는다.

경계는 한 방향으로만 열린다

패키지 내부에서 층은 눈에 보이게 갈라 둔다. `proto/`(와이어 계약), `proto-codec.ts`(인코딩/디코딩), `view/`(파생 API 계약). 규칙은 하나다: *`view/`는 와이어 층을 import 할 수 있고, 반대는 금지다.* 파생 방향을 코드로 강제한 것이다. 가능하면 파생을 기계적으로 표현한다. 예를 들어 proto 파생 타입에서 `idempotency_key` 를 빼고 `feed`, `stale_ms`, `stale_threshold_ms` 를 더하는 형태로 적으면, proto 에 필드가 늘 때 뷰가 자동으로 따라온다.

3.8 정리: 계약 하나가 지나가는 세 층

이 장에서 본 것을 한 문장으로 모으면 이렇다. *proto* 가 형태를 정하고, *codec* 이 그 형태를 바이트로 옮기고, *ClickHouse DDL* 이 그 형태를 저장한다. 그리고 이 순서를 지키는 한, 계약은 몇 개의 필드가 추가되어도 여전히 하나다. `venue` 필드 하나의 여정이 이것을 보여 줬다. 이 계약이 지켜지는 한, 같은 이벤트가 두 번 들어와도 항상 같은 값을 갖는다는 사실이 성립하고, 그 사실이 장 5 에서 다루는 중복 제거의 전제가 된다.

Chapter 4

델타에서 스냅샷을 재구성한다는 것

이 장은 거래소가 매번 전체 호가창을 보내지 않고 변경분 (delta) 만 보내는 이유, 그리고 그 변경분을 누적해 전체 스냅샷 (snapshot) 을 다시 만들어내는 재구성기 (reconstructor) 가 왜 WS 연결을 직접 쥔 producer 안에 있어야만 하는지를 다룬다. 이 장에서 정리하는 `BookEvent`/`VenueOrderbookAdapter` 경계는 장 5 에서 다루는 순서 보장 문제 (§4.4 의 last-write-wins 가드) 의 전제가 된다.

[**TODO**] §4.0–§4.6 을 바탕으로 재구성이 producer 에 남아야 하는 이유 (seq 갭 감지 시 reconnect 권한), Redis 라이터가 무상태인 이유 (A 안 채택 근거), venue 중립 어댑터 경계를 설명한다.

Chapter 5

at-least-once 에서 exactly-once 로

In brief.

Why 스트리밍 파이프라인은" 죽었다 다시 살아나는" 일이 늘 일어난다. 그때마다 메시지가 사라지거나 (유실) 두 번 들어오는 (중복) 일이 생기는데, 이 파이프라인이 감당해야 하는 진짜 문제는 유실을 완전히 없애는 것이 아니라 어디서 유실이 나는지를 정확히 알고, 그 나머지 구간에서는 중복을 안전하게 접는 것이다.

When producer 가 거래소 WS(웹소켓, WebSocket) 연결을 받아 Kafka 로 내보내는 순간부터, ClickHouse 의 데이터 테이블에 행이 최종적으로 쌓이는 순간까지 전 구간에 적용된다.

Where 이 장의 결론은 예측시장 (Kalshi, Polymarket) 과 현물 (Binance, Coinbase) 가족 모두에 동일하게 적용된다. `venue` 축 (장 3) 이 이 구간의 보장 수준을 바꾸지 않기 때문이다.

How 앱 내부 메모리 큐 구간은 보장이 아예 없다는 사실을 받아들이고, 그 뒤 구간 (브로커부터 ClickHouse 까지) 은 정렬 키 *(sort key)* 가 완전히 같은 행만 접는다는 ClickHouse `ReplacingMergeTree` 의 규칙 하나로 중복을 흡수한다.

5.1 배경: " 적어도 한 번" 과" 정확히 한 번" 이라는 말이 왜 미끄러운가

스트리밍 파이프라인을 처음 설계할 때 누구나 듣는 말이 있다. "at-least-once(적어도 한 번 전달) 로 받고 exactly-once(정확히 한 번) 로 저장한다." 문제는 이 두 단

어가 파이프라인 전체의 성질처럼 들리지만, 실제로는 구간마다 다르게 성립한다는 점이다. 한 파이프라인 안에 유실이 나는 구간과 중복만 나는 구간이 동시에 있을 수 있고, " 이 시스템은 at-least-once 다" 라는 한 문장은 그 차이를 지워 버린다.

이 파이프라인은 producer(거래소 WS 를 구독해 Kafka 로 내보내는 프로세스) 3 종에서 시작해 Kafka, 그리고 ClickHouse 의 데이터 테이블까지 이어지는 사슬이다. 이 장은 그 사슬을 네 구간으로 쪼개, 각 구간이 실제로 어떤 보장을 갖는지 하나씩 확인한다.

*at-least-once(*적어도 한 번 전달*)* 란" 메시지가 유실되지는 않지만, 같은 메시지가 두 번 이상 도착할 수 있다" 는 보장이다. 반대로 *exactly-once(*정확히 한 번*)* 는" 유실도 없고 중복도 없다" 는, 더 강하지만 훨씬 지키기 어려운 보장이다. 스트리밍 시스템에서 진짜 exactly-once 를 배달 단계에서 보장하는 방법은 사실상 없다시피 하다. 대신 이 파이프라인이 택한 길은" 배달은 중복을 허용하되, 저장 단계에서 중복을 접어 *effectively-exactly-once(*사실상 정확히 한 번*)* 를 만든다" 이다. 절 5.5 에서 그 방법을 다룬다.

5.2 앱 내부 큐: at-least-once 조차 아닌 구간

무슨 문제가 있었나. producer 가 거래소 WS 로부터 메시지를 받으면, 그 메시지를 곧바로 Kafka 로 보내지 않는다. 먼저 애플리케이션 안의 큐 (`BufferedKafkaProducer.queue`) 에 쌓아 두고, `BATCH_SIZE`(100 개) 또는 `FLUSH_MS`(2000ms, 2 초) 조건 중 먼저 차는 쪽에서 한꺼번에 드레인 (비워서 내보냄) 한다.

이 큐는 순수 메모리 *(in-memory)* 다. 디스크에 쓰지 않고, 다른 프로세스로 복제하지도 않는다. 그래서 producer 프로세스가 죽으면, 그 순간 큐 안에 있던 메시지는 그대로 사라진다. 최악의 경우 `FLUSH_MS` 가 다 차기 직전, 즉 최대 2 초분의 메시지가 한 번에 사라질 수 있다.

왜 이것이" 조용한" 유실인가

여기서 진짜 문제는 유실이 일어난다는 사실 자체가 아니다. 그 유실이 아무 흔적도 남기지 않는다는 점이다. 두 가지가 겹쳐서 이런 결과를 만든다.

1. `dr_cb: false`. *delivery report* 콜백은 Kafka 클라이언트 라이브러리 (librdkafka) 가" 이 메시지가 결국 브로커에 쓰였는지, 아니면 끝내 실패했는지" 를 앱에 알려 주는 통로다. 이 콜백이 꺼져 있으면, produce 가 최종적으로 실패해도 애플리케이션은 그 사실을 알 방법이 없다.

2. WS 피드는 재생 *(replay)* 이 없다. 거래소가"5 초 전에 보낸 그 메시지를 다시 보내 달라" 는 요청을 받아 주지 않는다. 한 번 놓친 메시지는 그 순간의 호가·체결 정보를 영원히 잃는 것과 같다.

왜 이것이" 최악의 종류" 의 버그인가. 버그에는 두 종류가 있다. 하나는 터지는 순간 로그나 에러 메시지, 크래시로 자기 존재를 알리는 버그다. 다른 하나는 아무 신호도 남기지 않고 그냥 결과가 조용히 틀려지는 버그다. 이 큐 유실은 후자다. 프로세스는 정상 종료 로그를 남기고, 재시작 후에도 아무 에러 없이 다시 동작한다. 잃어버린 2 초분의 데이터는 로그 어디에도, 메트릭 어디에도 나타나지 않는다. 유실이 있었다는 사실을 알아채려면 ClickHouse 에 쌓인 시계열에서 시간 구멍을 직접 찾아내는 수밖에 없는데, BTC 1 종처럼 메시지가 촘촘한 스트림에서는 그 구멍조차 눈에 띄기 어렵다. 발견되지 않는 버그는 고쳐지지도 않는다. 이것이" 발생을 완전히 막을 수는 없어도, 최소한 발생했다는 사실만은 반드시 알 수 있어야 한다" 는 원칙이 스트리밍 파이프라인 설계에서 특히 중요한 이유다.

여기에 셧다운 경합 (race) 하나가 더 있다. 프로세스가 종료 신호를 받은 뒤에도 아직 처리 중이던 메시지가 큐에 넣으려 시도하면, 이미 닫힌 연결이라 그 메시지는 버려지고 `warn` 로그 한 줄만 남는다 (`enqueue after disconnect dropped`). 이 경우는 앞의 두 경우와 달리 로그가 남기는 하지만, error 가 아니라 warn 이라 운영 관점에서 놓치기 쉽다.

정리하면, producer 내부 큐부터 Kafka 브로커에 데이터가 실제로 쓰이기까지의 구간은 *at-least-once* 조차 성립하지 않는다. " 적어도 한 번은 전달된다" 는 말 자체가 이 구간에는 적용되지 않으며, 그 사실이 로그에도 남지 않는다.

5.3 브로커부터 ClickHouse 까지: at-least-once 가 성립하는 구간

무슨 문제가 있었나. 메시지가 큐를 벗어나 Kafka 클라이언트 내부 버퍼로 넘어간 다음부터는 사정이 다르다. 여기서부터는 네트워크 문제나 브로커 일시 장애로 인한 실패를 재시도 *(retry)* 로 극복할 수 있는 구간이기 때문이다. 문제는" 재시도가 있다" 는 사실만으로는 안심할 수 없다는 것이다. 재시도가 있어도 ack 수준이 약하면 여전히 유실이 날 수 있고, 재시도가 만드는 중복을 받아줄 준비가 없으면 중복 자체가 새로운 문제가 된다.

성립하는 이유. librdkafka(Kafka 클라이언트 라이브러리) 는 ack(승인) 를 받지 못한 produce 요청을 `message.timeout.ms` 까지 자동으로 재시도한다. 이 재시도는 같은 바이트를 다시 보내는 것이므로, 같은 `idempotency_key`(멱등성 키) 와 같은 `time_ms` 를 가진 메시지가 그대로 다시 전송된다. 즉 여기서 생기는 중복은 전부 완전히 동일한 행이다. 이 성질이 절 5.5 의 중복 제거를 가능하게 만드는 전제가 된다.

ClickHouse 의 `ENGINE=Kafka` 테이블도 같은 원리로 동작한다. Kafka 엔진은 메시지를 읽어 *materialized view(*구체화 뷰*, MV)* 에 삽입을 마친 후에만 그 오프셋 (offset, 읽은 위치) 을 커밋한다. 삽입과 커밋 사이에서 ClickHouse 가 죽으

면, 재시작 후 같은 오프셋부터 다시 읽어 같은 블록을 다시 삽입한다. 이 역시 완전히 동일한 행의 중복을 만든다.

단서 하나: ack 수준. 코드는 `acks` 옵션을 명시하지 않으므로 librdkafka 의 기본값인 `request.required.acks = -1`(all, 모든 복제본의 ack 를 기다림) 이 적용된다. 그런데 이 파이프라인의 Kafka 클러스터는 *replication factor(*복제 계수*)* 가 *1* 이다. 복제본이 하나뿐이면" 모든 복제본의 ack" 는 결국" 리더 브로커 한 대의 ack" 와 같은 뜻이 된다. 즉 설정값은 `acks=all` 이지만 실질 효과는 ***`acks=1`*** 이다. 그 한 대가 디스크에 fsync(강제로 디스크에 씀) 하기 전에 죽으면, 재시도로도 메꿀 수 없는 유실이 난다. 이 단서는 절 5.7 에서" 정직하게 어디까지가 남은 위험인가" 를 정리할 때 다시 등장한다.

정리하면, 브로커부터 *ClickHouse* 데이터 테이블까지는 *at-least-once* 가 성립한다. 재시도와 지연 커밋이 만드는 중복은 전부 바이트 단위로 동일한 재전송이라는 성질을 가지며, 이 성질을 절 5.5 이 이용한다.

5.4 파이프라인 전체 지도

앞 두 절의 내용을 한 장의 그림으로 모으면 그림 5.1 와 같다. 화살표 왼쪽의 괄호 번호는 이 장에서 다룬 구간 번호이고, 화살표 위 라벨이 그 구간의 실제 보장 수준이다.

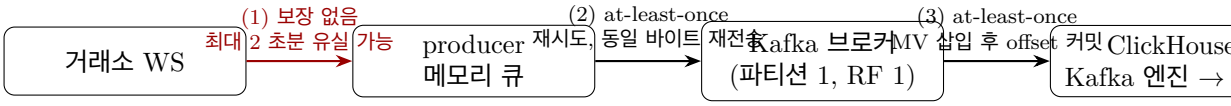


Figure 5.1: producer 큐부터 ClickHouse 데이터 테이블까지, 구간별 전달 보장 수준. (1) 만 보장이 없고 나머지는 at-least-once 이며, (4) 에서 effectively-exactly-once 로 접힌다.

5.5 DB 단에서 exactly-once 로 접는 방법

무슨 문제가 있었나. 절 5.3 에서 확인했듯, 브로커 이후 구간은 중복을 만들 수는 있어도 그 중복은 전부 바이트 단위로 동일한 재전송이다. 그렇다면 남은 문제는 하나다. 동일한 행이 여러 번 들어왔을 때, 저장 단계에서 그것을 자동으로 한 행으로 되돌릴 방법이 필요하다는 것이다.

그래서 무엇을 바꿨나. 데이터 테이블의 엔진으로 `ReplacingMergeTree(ingested_at)` 를 쓰고, 정렬 키 (sort key) 를 `ORDER BY (venue, ticker, time, idempotency_key)` 로 잡는다.

왜 그 변경이 문제를 푸는가. *ReplacingMergeTree* 는 백그라운드에서 파트 (part, 저장 단위) 를 병합 (merge) 할 때, 정렬 키가 완전히 같은 행들만 하나로 접는다. 여기서" 완전히 같다" 는 것은 `idempotency_key` 하나만 같아서는 안 되고, `venue`, `ticker`, `time` 까지 네 컬럼 전부가 같아야 한다는 뜻이다. 정렬 키의 일부만 같은 행은 별개의 행으로 남는다.

이 규칙이 이 파이프라인에서 실제로 작동하는 이유는, 절 5.3 에서 확인한 성질 때문이다. 이 파이프라인의 모든 중복은 바이트 단위로 동일한 재전송이므로, 같은 이벤트가 두 번 들어오면 `venue`, `ticker`, `time`, `idempotency_key` 네 값이 매번 똑같다. 여기서 `time` 은 ClickHouse 가 메시지를 받은 시각이 아니라, MV 가 페이로드의 `time_ms` 를 `toDateTime64(time_ms/1000, 3)` 으로 변환해 얻은 값이다. 즉 *payload* 안의 값에서 나온다. 재전송이 일어나도 페이로드 자체는 바뀌지 않으므로 `time` 도 매번 같고, 결과적으로 정렬 키 네 컬럼 전부가 자동으로 일치해 병합 시 접힌다.

이 불변식을 깨는 유일한 경우

정렬 키가 완전히 같아야 접힌다는 규칙은 거꾸로, 애플리케이션이 같은 이벤트를 다른 *time_ms* 로 다시 만들어 내면 중복 제거가 작동하지 않는다는 뜻이기도 하다. 예를 들어 producer 가 재시작한 뒤, 재시도로 보낸 것이 아니라 같은 오더북 상태를 새로운 *Date.now()* 로 다시 *publish* 하면, 그 두 행은 `time` 이 달라 정렬 키가 일치하지 않고 이력에 별개의 두 행으로 남는다. 이것은 재전송이 아니라 새로운 이벤트로 취급되는 것이므로, ReplacingMergeTree 입장에서는 정상 동작이다.

`venue` 가 정렬 키 선두에 있어야 하는 이유. Kalshi 의 티커는 `KXBTCD-26SEP22-B112000` 처럼 시리즈 형태 문자열이고, Polymarket 의 티커는 CLOB(Central Limit Order Book) token id 로, 수십 자리 10 진수 문자열이다. 두 거래소의 티커 값이 우연히 같은 문자열이 되는 일은 실무적으로 없다고 봐도 되지만, 그 가능성을 완전히 배제하려면 `venue` 가 정렬 키와 `idempotency_key` 양쪽 모두에 들어가야 한다.

`venue` 가 없다면, 두 거래소의 행이 `ticker` 만으로 구분되므로, 티커 문자열이 우연히 겹치는 극단적인 경우 서로 다른 거래소의 행이 하나로 잘못 접힐 수 있다. `venue` 를 정렬 키 선두에 두면 이 위험이 원천적으로 사라진다. 동시에 `venue` 는 카디널리티 (값의 종류 수) 가 2 4 개로 낮아, ClickHouse 의 스파스 인덱스 (sparse index) 가 `WHERE venue='kalshi'` 조건을 인덱스 단계에서부터 효율적으로 잘라낼 수 있다는 부수적인 이득도 있다.

예제 5.1 (venue 가 없었다면 벌어졌을 일) 가상의 상황을 생각해 보자. *Kalshi* 티커 *KXBTCD-26SEP22-B112000* 과 *Polymarket* 의 어떤 *CLOB token id* 가 *(*실제로는 형태가 전혀 달라 일어나지 않지만*)* 문자열로 우연히 같다고 가정하면, *venue* 없이 *ORDER BY (ticker, time, idempotency_key)* 만 쓸 때 두 거래소의 서로 다른 오더북 행이 같은 *time*, 같은 *idempotency_key* 를 우연히 공유하

는 순간 하나로 접혀 버린다. 서로 다른 시장의 데이터가 뒤섞이는 것이다. *venue* 를 선두에 추가하면 *(kalshi, KXBTCD-...)* 와 *(polymarket, KXBTCD-...)* 는 애초에 다른 정렬 키를 갖게 되어 이 위험이 사라진다.

5.6 왜 Kafka 트랜잭션 (EOS) 이 아니라 DB 중복 제거인가

무슨 문제가 있었나. Kafka 는 exactly-once semantics(EOS) 라는 자체 트랜잭션 기능을 제공한다. " 그러면 그걸 쓰면 되지 않느냐" 는 질문이 자연스럽게 나온다.

그래서 무엇을 바꿨나. 쓰지 않기로 했다. 결정적인 이유는 하나다. *ClickHouse* 의 *ENGINE=Kafka* 테이블은 트랜잭션 컨슈머 *(transactional consumer)* 가 아니다. Kafka EOS 는 트랜잭션을 지원하는 컨슈머 위에서만 의미가 있는데, 이 파이프라인의 읽기 쪽 (ClickHouse Kafka 엔진) 이 애초에 그 조건을 만족하지 못한다.

왜 그 변경이 문제를 푸는가. DB 단 중복 제거는 Kafka EOS 보다 적용 범위가 더 넓다. Kafka EOS 는 Kafka 내부의 read-process-write 패턴만 커버하지만, DB 단 중복 제거는 producer 크래시, 브로커 재시도, 컨슈머 재읽기까지 경로에 상관없이 전부 커버한다. 또한 producer 가 재시작해 PID(프로세스 ID) 가 바뀌어도 `idempotency_key` 가 안정적으로 재계산되는 한 중복 제거가 그대로 유지된다. Kafka EOS 는 PID 가 바뀌면 트랜잭션 보장이 사라진다.

	DB 중복 제거 (채택)	Kafka 트랜잭션 EOS
적용 범위	producer 크래시, 브로커 재시도, 컨슈머 재읽기 전부	Kafka 내부 read-process-write 만
ClickHouse Kafka 엔진	지원	트랜잭션 컨슈머 미지원. 애초에 쓸 수 없음
producer 재시작 후	`idempotency_key` 가 안정적이면 중복 제거 유지	PID 가 바뀌면 보장 소멸
비용	병합 시 CPU, 읽기 시 `FINAL`	처리량 저하, 운영 복잡도

Table 5.1: DB 단 중복 제거와 Kafka 트랜잭션 EOS 비교.

5.7 정직하게 짚는 잔존 위험

무슨 문제가 있었나. 지금까지의 논의를" 이 파이프라인은 exactly-once 다" 로 요약하면 과장이다. *effectively-exactly-once* 라는 표현을 쓴 이유는, 그 보장이 성립하지 않는 지점이 실제로 남아 있기 때문이다. 이 절은 그 지점을 숨기지 않고 나열한다.

(1) 중복이 아니라 유실로 바뀌는 지점: `kafka_skip_broken_messages = 1`. ClickHouse Kafka 엔진은 이 설정 때문에, 들어온 메시지가 proto 스키마와 맞지 않으면 그 행을 그냥 버린다. 이것은 지금까지 이야기한" 중복" 과 성질이 다르다. 중복은 나중에 접히기라도 하지만, 이 경우는 처음부터 유실이다. 스키마를 바꿀 때 (장 3 의 3 단계 계약) 가 이 파이프라인 전체에서 가장 위험한 순간인 이유가 여기에 있다. proto, codec, DDL 세 단계의 순서를 지키는 규율이 이 위험을 막는 유일한 수단이다.

(2) `acks=all` 이 사실상 `acks=1` 인 지점. 절 5.3 에서 다룬 것처럼, replication factor 가 1 이라 `acks=all` 의 실질 효과는 `acks=1` 이다. 재시도는 있지만, 그 재시도가 기대는 브로커가 단 한 대뿐이라는 사실은 바뀌지 않는다. 그 한 대의 디스크가 완전히 망가지면, 재시도로도 메꿀 수 없는 유실이 난다.

(3) 순서가 보장되지 않는 지점. Kafka 토픽은 전부 파티션이 1 개이므로 얼핏 순서가 유지될 것 같지만, 실제로는 그렇지 않다. producer 가 `enable.idempotence: false` 로 설정되어 있고, librdkafka 의 `max.in.flight.requests.per.connection`(연결당 동시 전송 가능한 요청 수) 기본값이 작지 않아, 재시도된 배치가 원래보다 늦게 도착할 수 있다. 파티션이 1 개라는 사실은 이 문제를 막아 주지 않는다.

ClickHouse 의 이력 테이블은 `time` 기준으로 정렬해서 읽으므로 순서 뒤집힘이 무해하다. 문제는 Redis 에 저장하는 현재 상태다. 오더북 라이터가 오래된 스냅샷을 나중에 받으면, 손대지 않을 경우 최신 상태를 오래된 값으로 덮어써 버린다. 그래서 Redis 쓰기에는 별도의 방어가 필요하다. `seq`(시퀀스 번호) 가 아니라 `time_ms` 를 비교하는 *last-write-wins(LWW,* 나중에 쓴 값이 이김*)* 가드를 Lua 스크립트로 구현해, 기존에 저장된 `time_ms` 보다 작거나 같은 스냅샷은 무시한다. `seq` 가 아니라 `time_ms` 를 쓰는 이유는 `seq` 가 재접속 시 리셋될 수 있는 값이기 때문이다. 이 가드는 `ob:{venue}:{ticker}` 단위로 걸리므로, 한 거래소의 순서 뒤집힘이 다른 거래소의 상태에 영향을 주지 않는다.

(4) 중복 제거는 최종적 (eventual) 이다. ReplacingMergeTree 의 병합은 백그라운드에서 비동기로 일어난다. 병합이 일어나기 전에는 같은 이벤트의 중복 행이 그대로 조회에 노출된다. 이 파이프라인은 BTC 1 종의 데이터량에서 `SELECT ... FROM t FINAL`(병합을 강제로 앞당겨 읽는 옵션) 로 이 문제를 단순하게 해결한다. 대시보드의" 현재 오더북" 은 애초에 ClickHouse 가 아니라 Redis 에서 읽으므로 이 지연의 영향을 받지 않는다.

5.8 한 줄로 보는 경로별 요약

표 5.2 는 이 장에서 다룬 네 구간을 한 표로 모은 것이다.

구간	보장 수준	근거
producer 큐 → Kafka 브로커	보장 없음. 최대 2 초분 조용히 유실 가능	메모리 큐, `dr_cb: false`, WS 는 재생 불가
Kafka 브로커 → ClickHouse Kafka 엔진	at-least-once	librdkafka 재시도, MV 삽입 후 offset 커밋
ClickHouse Kafka 엔진 → 데이터 테이블	중복은 발생하되 전부 바이트 동일	재시도/재읽기가 같은 payload 를 반복
데이터 테이블 병합	effectively-exactly-once (최종적)	`ReplacingMergeTree` + 정렬 키 완전 일치

Table 5.2: producer 큐부터 ClickHouse 데이터 테이블까지, 구간별 전달 보장 요약.

예제 5.2 (숫자로 보는 세 가지 시나리오) 같은 *Kalshi* 오더북 이벤트 하나 *(venue=kalshi, ticker=KXBTCD-26SEP22-B112000, time_ms=1758500000123, seq=4711)* 를 예로 세 가지 경우를 비교한다.

***(a)* 브로커 *ack* 유실 후 재시도, 그리고 삽입-커밋 사이 크래시.** 같은 바이트가 *producer* 에서 두 번, *ClickHouse* 에서 다시 두 번 재처리되어 데이터 테이블에 임시로 *4* 개의 행이 쌓일 수 있다. 네 행 모두 *(venue, ticker, time, idempotency_key)* 가 동일하므로, 병합 후에는 *1* 개 행만 남는다.

***(b) producer* 재시작 후 같은 오더북 상태를 새 시각으로 재발행.** 이것은 재시도가 아니라 새 이벤트 생성이다. 새 *time_ms* 를 얻으므로 정렬 키가 달라지고, 이력에는 *2* 개의 서로 다른 행으로 남는다. 이것이 절 *5.5* 에서 말한 *"* 불변식을 깨는 유일한 경우*"* 의 실제 사례다.

***(c)* 같은 티커 문자열을 가진 두 거래소.** *venue* 가 다르므로 *Redis* 키는 *ob|kalshi|KXBTCD-26SEP22-B112000|4711* 과 *ob|polymarket|KXBTCD-26SEP22-B112000|4711* 로 처음부터 갈라지고, *ClickHouse* 정렬 키도 *venue* 컬럼이 다르므로 두 거래소의 행이 병합에서 하나로 접히는 일은 없다.

정리. producer 메모리 큐부터 브로커까지는 at-least-once 조차 아니다. 크래시가 나면 최대 2 초분이 조용히 사라지고, `dr_cb: false` 때문에 그 유실은 로그에도 남지 않는다. 브로커부터 ClickHouse 데이터 테이블까지는 at-least-once 이며, 그 구간의 모든 중복은 바이트 단위로 동일한 재전송이므로 `idempotency_key` 를 정렬 키에 포함한 `ReplacingMergeTree` 가 effectively-exactly-once 로 접는다. 다만 순서는 보장되지 않으므로 (idempotence off), Redis 의 현재 상태는 `time_ms` 기준 last-write-wins 가드로 별도로 방어한다.

Chapter 6

예측과 트레이딩

이 장은 지금까지 쌓은 데이터 (오더북 이력, 체결, 틱) 를 실제 예측·트레이딩 로직에 연결하는 단계를 다룬다. MVP1 은 데이터 파이프라인의 형태만 증명하므로, 이 장의 본문은 그 파이프라인 위에 무엇을 얹을 것인가가 실제로 설계된 뒤에 채운다.

[**TODO**] 예측/트레이딩 설계가 나오면 작성. 현재 00.architecture.mvp1.md 는 이 영역을 명시적으로 범위 밖 (§1 " 제외" 목록의 arbitrage/fair-value/Kelly/주문집행) 으로 두고 있으므로, 이 장은 그 설계 문서가 나온 뒤 채운다.

Bibliography

[1] system-architect. 00. architecture — mvp1 (btc data pipeline, mock-first). 00.architecture.mvp1.md. project_market_predict, 2026-09-22.